

CS4475 Fall 2026
Diego A Guerrero
9/24/26
daguerrero9@miners.utep.edu

Lab Report
Submitted on

PA 1: Implementing xv6 User Commands and System Calls
Repository: <https://github.com/MokeyDX/xv6-riscv>
Branch: pa1-syscalls

Task 1. (5 pts) Implement the uptime utility

user/uptime.c invokes the existing uptime() function system call, prints the number of clock ticks since boot, and calls exit(0). And line \$U/_uptime was added to UPROGS.

```
$ uptime  
up 167 clock ticks
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

user/time1.c validates that a command was supplied, records uptime() before fork(), uses exec(argv[i], &argv[i]) in the child, uses wait() in the parent, prints the difference between ending and starting ticks.

```
$ time1 sleep 10  
elapsed time: 10 ticks
```

```
$ time1 matmul  
Time: 5 ticks  
elapsed time: 5 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

Files changed:

- kernel/proc.h: Added uint cputime to struct proc.
- kernel/proc.c: Initialized cputime to zero in allocproc().

- kernel/trap.c: Incremented the running process's cputime on timer interrupts in both usertrap() and kerneltrap() before yielding.
- Makefile: Changed CPUS := 3 to CPUS := 1.

A process earns one CPU-time tick when its running as a timer interrupt occurs.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

Files changed:

- kernel/pstat.h: Defines struct rusage containing uint cputime.
- user/user.h: Declares struct rusage and wait2().
- user/usys.pl: Generates the user-space wait2 syscall stub.
- kernel/syscall.h: Assigns syscall number 23.
- kernel/syscall.c: Registers sys_wait2.
- kernel/sysproc.c: Retrieves the two user pointers and calls kwait2().
- kernel/defs.h: Declares kwait2().
- kernel/proc.c: Implements kwait2() using the same locking and child-search logic as kwait(). It copies the exit status and rusage to the parent before freeing the child.

copyout() is necessary because the kernel cannot directly dereference arbitrary user-space pointers.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

user/time.c follows the same fork()/exec() structure as time1, but calls wait2() and calculates: CPU percentage = cputime × 100 / elapsed time

The zero-elapsed-time check that prevents division by zero.

```
$ time sleep 10
```

```
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
```

```
$ time matmul
```

```
Time: 5 ticks
```

```
elapsed time: 5 ticks, cpu time: 5 ticks, 100% CPU
```

```
$ time matmul & time matmul &
```

```
Time: 9 ticks
```

```
elapsed time: 10 ticks, cpu time: 5 ticks, 50% CPU
```

Time: 8 ticks

elapsed time: 9 ticks, cpu time: 5 ticks, 55% CPU

The sleeping process consumes no complete CPU ticks, one CPU-bound process uses the CPU continuously, and two CPU-bound processes share the single configured CPU approximately equally.

The complete xv6 usertests regression suite finished with ALL TESTS PASSED.

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

In this assignment, I learned how user commands, processes, timer interrupts, and system calls work together in xv6. The uptime and time1 commands showed me how a user program can call existing kernel services and use fork(), exec(), and wait() to run another program. I also learned the difference between elapsed time and CPU time. Elapsed time includes all the time between starting and finishing a command, including time spent sleeping or waiting. CPU time counts only the timer ticks during which the process was actually running.

Implementing wait2() helped me understand the complete path of an xv6 system call. The function had to be declared in user/user.h, added to user/usys.pl, assigned a number in kernel/syscall.h, registered in kernel/syscall.c, handled by sys_wait2() in kernel/sysproc.c, and implemented by kwait2() in kernel/proc.c. I learned that the kernel uses copyout() to safely copy the child's exit status and resource-usage information into the parent's user address space. I also learned why this information must be copied before freeproc() clears the child's process structure.

One difficulty was coordinating changes across many files. A missing declaration, syscall-table entry, or generated stub would prevent the system call from working. I addressed this by making the changes in small stages, compiling after each stage, and checking each connection individually. Another difficulty was placing pstat.h after types.h, because struct rusage uses the kernel's uint type. I corrected the include order before rebuilding.

Testing concurrent processes was also challenging because their terminal output was interleaved. I initially mistyped one concurrent test while another command was still producing output. I reran the test carefully and obtained approximately

equal CPU sharing: one process received 50% CPU and the other received 55%. Finally, I ran the complete xv6 usertests suite, which ended with ALL TESTS PASSED. This gave me confidence that the new process-accounting field and wait2() system call did not break existing xv6 behavior.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One important limitation is that this implementation reports CPU time only for the direct child created by the time command. If the timed program creates additional child processes, wait2() returns only the CPU time stored in the original child's struct proc. It does not add the CPU time consumed by grandchildren or other descendant processes. As a result, a program that delegates most of its work to child processes could appear to use very little CPU even though its entire process family used a significant amount.

The measurements also have limited precision because xv6 measures time in timer ticks. A program that starts and finishes between two timer interrupts may report zero elapsed ticks and zero CPU ticks. For short programs, a difference of one tick can change the calculated percentage substantially. The CPU percentage also uses integer division, so fractional percentages are discarded rather than rounded precisely.

The elapsed time includes overhead from creating the child, executing the command, waiting for it, and returning through the system call. Therefore, it is not a perfect measurement of only the target program's work. The implementation also combines user-mode and kernel-mode execution into a single CPU-time value, unlike more advanced implementations of /bin/time that report user and system CPU time separately.

The CPU-time counter can eventually overflow because it is stored as an unsigned integer. In addition, the expected CPU percentages in this assignment depend on xv6 running with one CPU. Finally, concurrent commands can write to the console simultaneously, causing their output to appear mixed together even though the timing calculations themselves are correct.

AI Disclosure:

I used OpenAI Codex as an interactive guide while completing this assignment. It helped me interpret the requirements, identify the xv6 files involved in each task, propose code structures and terminal commands, review compiler and runtime output, and diagnose typing and formatting mistakes. I verified the final behavior rather than submitting untested generated code.